

Faculty Teaching & Office-Hours Schedule Generator

COSC 631 – Software Engineering · Second Project Report

Carlson Cox
September 2026

Contents

- [1. Overview](#)
- [2. System design](#)
- [3. Input data](#)
- [4. Static libraries](#)
- [5. The Makefile](#)
- [6. Scheduling logic](#)
- [7. Error handling and robustness](#)
- [8. Input file location: local vs. Internet](#)
- [9. Testing](#)
- [10. Sample output](#)
- [11. Limitations and future work](#)
- [Appendix A – Submission contents](#)

1. Overview

This project is a C program, `schedgen`, that automatically generates each professor's **teaching schedule** and **office-hours schedule** for a faculty. It reads **exactly two input files**:

1. **Class schedule**: the department's CTEC Fall 2026 schedule (56 sections, 25 instructors), exactly as distributed as an Excel workbook. It holds section IDs, titles, credits, days, times, rooms, instructors and enrollment. The sections run in two 7-week sessions, **7R1** and **7R2**.
2. **Faculty information file**: one record per instructor with department, office, contact details, courses taught, office-hours requirements, and every spelling of the instructor's name that appears in the schedule.

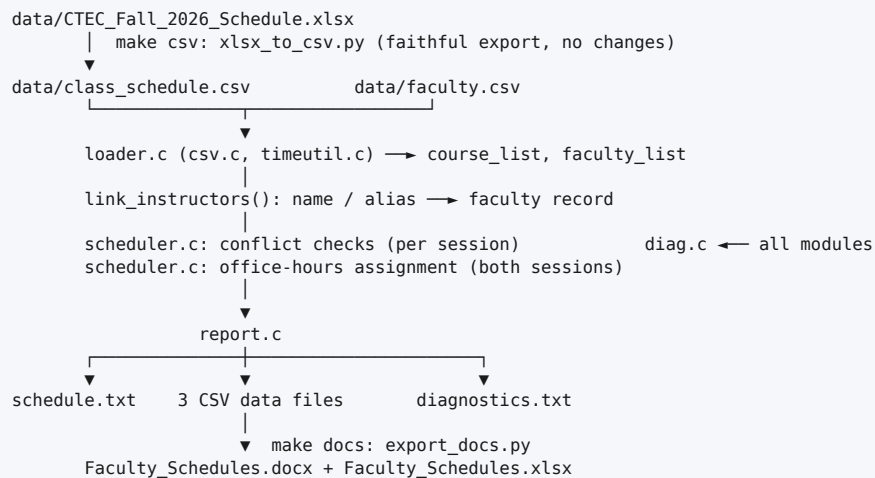
The program validates both files against each other, detects scheduling conflicts, repairs obvious data-entry errors (with a warning), and assigns office hours that never fall during class time. It writes a formatted text report plus CSV data. Two Python scripts, called from the Makefile, export the department's `.xlsx` to CSV before the C step and convert the C output into **Word** and **Excel** documents in the style of the sample tables in the assignment. All scheduling logic lives in a **static library**, `libschedule.a`; `main.c` is a thin driver linked against it.

1.1 Requirements traceability

Requirement (from the assignment)	Where it is satisfied
Implemented in C	src/main.c, src/lib/*.c, include/*.h (C11, ~3,700 lines)
Exactly two input files	main.c rejects any other count (exit status 1); inputs are the class schedule and data/faculty.csv
Syllabus / class-schedule file	The department's CTEC Fall 2026 schedule, read in its native column layout (§3)
Faculty information file	data/faculty.csv: lecturers, department, courses taught, office-hours policy (§3.2)
Generate teaching + office-hours schedules	scheduler.c (assignment), report.c (tables), §6
Use office-hours tables provided by the lecturer	fixed_office_hours column, validated and kept unless it collides with a class (§6.2)
Sample tables are the <i>style</i> of output, not hard-coded data	Nothing is hard-coded; output mirrors the sample's Course / Times / Location layout (§10)
Organize the project into modules	7 library modules + driver, each with its own header (§2)
Create and use a static library	build/libschedule.a via ar rcs, linked with -Lbuild -lschedule (§4)
Makefile: compile, create library, link, generate output	Makefile targets all, lib, csv, run, docs, test (§5)
Python from the Makefile for Word/Excel	make csv → xlsx_to_csv.py; make docs → export_docs.py (§5.5)
Explain Internet vs. local input files	§8
Submit source, headers, library, Makefile, inputs, Python, sample output, README	Appendix A

2. System design

2.1 Architecture



2.2 Modules

Module	Header	Responsibility
common.c	common.h	Size limits, status codes, bounded string copy/trim, case-insensitive compare, dynamic-array growth
timeutil.c	timeutil.h	Parse 4:00 PM, 16:00, 4:00:00 PM (Excel), MW, TR, TTh; format "Monday & Wednesday 4:00 PM to 6:50 PM"
csv.c	csv.h	RFC 4180 reader/writer; header matching that ignores case, spaces and punctuation, with prefix match (Instructor*)
diag.c	diag.h	Collects INFO / WARNING / ERROR messages so modules never print directly
loader.c	model.h	Loads both files; splits CTEC 350.170 into course and section; extracts the session; classifies meetings as scheduled / asynchronous / TBA; repairs AM/PM errors; resolves instructor names through aliases
scheduler.c	scheduler.h	Session-aware conflict detection (instructor, split rooms); office-hours assignment on a 15-minute weekly grid
report.c	report.h	Text report (tables, data notes, weekly grid), CSV data files, diagnostics file
main.c	-	Command-line parsing, orchestration, exit codes. Not in the library

include/schedule.h is an umbrella header: a client needs only `#include "schedule.h"` and `-lschedule`. The unit-test program is a second client of the same library, showing the library is reusable without `main.c`.

2.3 Key design decisions

- **Read the department's format as-is.** Columns are found by header name, so Instructor 2025, CR and End (trailing spaces) need no hand editing, and column order does not matter.
- **Repair in C, not in Python.** The `.xlsx` → CSV export changes no values. Every correction happens in the C loader and is reported, so the same rules apply whether the CSV came from the script or from Excel's "Save As".
- **Report, don't refuse.** A conflict in the department's data is a finding, not a crash. All schedules are still produced, problems are listed first, and the exit status (3) lets scripts detect them.

- **The C program owns all formatting.** The CSV output carries display strings, so the Python exporter only lays them out, and the text, Word and Excel outputs can never disagree.

3. Input data

3.1 The class schedule (input 1)

Column	Example	Interpretation
ID	CTEC 350.170	Course CTEC 350, section 170
Course Title	Prin & Meth of Intru Det & Pre (7R1)	Title; trailing (7R1) = session (first 7 weeks)
CR	3	Credits
Days	MW, TR, MWF	R = Thursday; blank = asynchronous or TBA
Begin, End	4:00 PM, 6:50 PM	Excel time values, exported as displayed
Room	108, 201/112, Online, Dept	Split room = both rooms; Online/Dept are not physical rooms
Instructor 2025	Adedoyin, Anthony	Matched to a faculty record by name or alias
Enroll #	22	Enrollment

3.2 The faculty file (input 2)

The schedule contains 28 distinct instructor strings for 25 people, in two name orders (Adedoyin, Anthony vs. Ruth Agada) and with variants (Bemley, Bemley, J, Bemley, Jesse; Eugene Harris, Eugene T. Harris). The faculty file gives each person a stable ID (CT01-CT25) and an aliases column listing every spelling. Matching ignores case, spacing and punctuation. An instructor string with no match is an ERROR, so a new or misspelled name can never silently become a separate professor.

The file also carries the office-hours policy per professor (required hours, block length, preferred days, availability window, lecturer-supplied hours). Office, email and phone are not in the department schedule and are left blank for the department to complete. The output then shows “Office TBA”, or “Online (virtual)” for instructors whose sections are all online.

3.3 Findings in the CTEC Fall 2026 data

The program reported the following. Each was checked by hand against the spreadsheet.

Severity	Finding	Program action
ERROR	CTEC 711.555 meets MWF 5:00 PM to 5:00 PM (zero length)	Listed as TBA; not placed on the grid
WARNING	CTEC 721.555 meets 7:00 PM to 9:50 AM	End read as 9:50 PM (matches the 7:00–9:50 PM pattern of the program)
WARNING	CTEC 120.170 meets 12:00 PM to 3:15 AM	End read as 3:15 PM
ERROR	Room 112 , 7R1, MW: CTEC 120.171 (rooms 201/112, 5:00–7:00 PM) overlaps CTEC 294.170 (4:00–6:50 PM)	Reported; both schedules produced
ERROR	Room 111 , 7R2, MW: CTEC 345.180 (5:00–7:50 PM) overlaps CTEC 450.180 (7:00–9:50 PM)	Reported
ERROR	Ruth Agada , 7R2, MW 7:00–9:50 PM: assigned to both CTEC 402.180 and CTEC 435.180	Reported

Not reported, correctly: the many instructors who teach the same time slot and room in both sessions (e.g. CTEC 350.170 in 7R1 and 350.180 in 7R2, both MW 4:00–6:50 PM in room 108). These are the same slot in different halves of the term, not a conflict. On this data a session-unaware check reports 27 double-bookings (11 instructor, 16 room); only 3 are real, so 24 would be false alarms.

4. Static libraries

4.1 What a static library is

A **static library** (Unix extension `.a`, “archive”) is a single file that bundles several compiled object files (`.o`) together with a **symbol index**. At link time the linker copies the machine code it needs from the archive directly into the executable. After linking, the program no longer depends on the library file at all.

This contrasts with a **shared (dynamic) library** (`.so` on Linux, `.dll` on Windows), which is loaded at *run time* and shared by every program that uses it.

	Static library (<code>.a</code>)	Shared library (<code>.so</code>)
When code is bound	Link time, copied into the executable	Run time, loaded by the dynamic loader
Executable size	Larger (contains library code)	Smaller
Deployment	Self-contained, nothing extra to install	Library must be present on the target machine
Updating the library	Relink every program	Replace the <code>.so</code> ; programs pick it up
Symbol resolution	Linker pulls only the object files that are needed	Whole library mapped

A static library is the right choice here: the program runs on a lab computer, must be easy to build and grade, and gains nothing from sharing code with other programs at run time.

4.2 How the library is created

```
# 1. Compile each library source into an object file (-c = compile only, do not link)
gcc -Iinclude -std=c11 -Wall -Wextra -c src/lib/timeutil.c -o build/obj/lib/timeutil.o
# ... same for common.c csv.c diag.c loader.c report.c scheduler.c

# 2. Archive the objects into the static library
ar rcs build/libschedule.a build/obj/lib/*.o
```

ar flags: **r** inserts or replaces members, **c** creates the archive without a warning, **s** writes the symbol index (equivalent to running ranlib), which lets the linker find which member defines a symbol.

make inspect shows the result:

```
== Members of build/libschedule.a (ar -t) ==
common.o csv.o diag.o loader.o report.o scheduler.o timeutil.o

== Exported functions (nm --defined-only, type T) ==
T load_courses      T link_instructors      T sched_check_conflicts
T load_faculty      T faculty_match      T sched_assign_office_hours
T report_write_text T report_write_csv    T tu_parse_time      ... (59 functions)
```

4.3 How the library is linked

```
gcc -Iinclude -c src/main.c -o build/obj/main.o      # compile the driver
gcc build/obj/main.o -Lbuild -lschedule -o build/schedgen # link against the library
```

- **-Iinclude**: where the **compiler** finds the headers. Headers declare the functions; the library defines them.
- **-Lbuild**: adds build/ to the **linker's** library search path.
- **-lschedule**: link libschedule.a (the linker adds the lib prefix and .a suffix).

Order matters. The linker processes inputs left to right, keeping a list of undefined symbols. When it reaches an archive, it extracts only members that resolve symbols already on that list. With the library *before* main.o, nothing is undefined yet, nothing is extracted, and the link fails:

```
$ gcc -Lbuild -lschedule build/obj/main.o -o schedgen
/usr/bin/ld: build/obj/main.o: in function `main':
main.c:82: undefined reference to `sched_config_default'
```

The linker map (-Wl,-Map=...) for the correct order shows which members were pulled in and why:

```
Archive member included to satisfy reference by file (symbol)
build/libschedule.a(diag.o)      build/obj/main.o (diag_init)
build/libschedule.a(loader.o)    build/obj/main.o (course_list_init)
build/libschedule.a(report.o)    build/obj/main.o (report_write_text)
build/libschedule.a(scheduler.o) build/obj/main.o (sched_config_default)
build/libschedule.a(timeutil.o)  build/libschedule.a(loader.o) (tu_parse_time)
build/libschedule.a(common.o)    build/libschedule.a(loader.o) (sched_copy)
build/libschedule.a(csv.o)       build/libschedule.a(loader.o) (csv_open)
```

ldd build/schedgen lists only the C runtime (libc.so.6), with no run-time dependency on libschedule.

5.4 Incremental builds (observed)

Change	What make did
Nothing	make: Nothing to be done for 'all'.
Edit src/lib/timeutil.c	Recompiled timeutil.o only → re-archived library → relinked program
Edit include/model.h	Recompiled only the files that include it (found via the .d files) → re-archived → relinked
Replace the .xlsx, then make docs	No C compilation; re-exported CSV → re-ran schedgen → re-ran the Python exporter

5.5 Python steps

make docs depends on the schedule stamp, which depends on the program, the exported CSV (which depends on the workbook) and the faculty file. One command therefore runs only the stages that are out of date, in order:

```
python3 scripts/xlsx_to_csv.py data/CTEC_Fall_2026_Schedule.xlsx data/class_schedule.csv
./build/schedgen -o output -b 15 data/class_schedule.csv data/faculty.csv
python3 scripts/export_docs.py --input output --docx output/Faculty_Schedules.docx --xlsx output/Faculty_Schedules.xlsx
```

Both scripts use openpyxl / python-docx (requirements.txt; make venv installs them). The exported CSV is shipped, so the C build and make run work even on a machine without those packages.

6. Scheduling logic

6.1 Weekly grid and sessions

For each professor, the week is 7 days × 96 slots of 15 minutes, each FREE, CLASS, BUFFER or OFFICE:

1. Every scheduled meeting from **both** 7R1 and 7R2 is marked **CLASS**. One office-hours schedule is then valid for the entire semester, and students see the same hours all term.
2. BUFFER minutes (default 15) before and after each class are protected. This enforces *“Instructors are not to be interrupted during class times.”*
3. Asynchronous and TBA sections occupy no time.

6.2 Lecturer-supplied office hours

Entries in fixed_office_hours (e.g. MW 3:00 PM-4:00 PM) are processed first. Those overlapping a class are **dropped** with a warning. Those only inside the buffer are **kept** (the lecturer’s choice) with an info message. The CTEC faculty file ships with this column empty; the end-to-end test exercises it.

6.3 Generated office hours

Required weekly hours: oh_hours_per_week, or the policy default **max(3 hours, 1 hour per section)**. Preferred days default to the days the professor already teaches. This matters for a program where

most sections meet in the evening and many instructors are on campus only two days a week. Each candidate block inside the availability window is scored:

Rule	Score	Rationale
Day on which the professor teaches	+30	Already on campus
Block touches a class buffer or an existing office-hours block	+20	"Office hours before class"; contiguous blocks
Each block already on that day	-50	Spreads hours across teaching days
Distance from that day's classes	-1 per 10 min	Keeps hours next to class, not 6 hours earlier
Day not preferred	-100	Used only if preferred days are full

The best block is placed; ties go to the earliest day and time, so output is deterministic. Touching generated blocks merge into one row. Example: Anthony Adedoyin teaches MW 4:00–6:50 PM (both sessions) and 7:00–9:50 PM (7R1), so 3 hours are required. The result is **Monday 1:45–3:45 PM** and **Wednesday 2:45–3:45 PM**, each ending at the 15-minute buffer before his 4:00 PM class.

6.4 Conflict detection

Check	Severity
Instructor teaching two overlapping sections in the same session	ERROR
Two sections sharing a physical room (including either half of 201/112) at overlapping times in the same session	ERROR
Instructor text not found in the faculty file	ERROR
Impossible time, invalid days, duplicate section ID	ERROR (section kept as TBA or skipped)
End time before start with AM end (data-entry error)	WARNING (corrected)
courses_taught disagrees with the schedule; lecturer hours overlap a class; requirement cannot be met	WARNING

7. Error handling and robustness

- Every library function returns a sched_status; allocation failures propagate to main (exit status 4).
- All string copies are bounded (sched_copy, snprintf); no strcpy/sprintf.
- Both input files are loaded before stopping, so a single run reports every input problem.
- Exit codes: 0 success, 1 usage, 2 input file unreadable, 3 data conflicts present, 4 output error.

8. Input file location: local vs. Internet

Decision: both input files are stored locally on the laboratory computer in the project's data/ directory. The C program opens them with standard C file I/O (fopen) using the two paths given on the command line.

Reasons:

1. **Reproducibility and grading.** The same inputs always produce the same output. A remote schedule is updated by the department during registration and could change between the student's run and the instructor's.
2. **No network dependency.** Lab machines may be offline, behind a proxy, or restrict outbound traffic.
3. **Simplicity and portability of the C code.** Standard C has no networking; downloading inside the program would require an external library such as libcurl, unrelated to the project's goals.
4. **Privacy.** The faculty file holds names and contact details; keeping it local avoids publishing it.
5. **Separation of concerns.** Where data comes from is independent of how it is processed; the program takes file paths, so any source works.

Internet option. If the department publishes the files online, the Makefile has an optional target that downloads them into data/ with curl, after which the normal pipeline runs unchanged:

```
make fetch SCHEDULE_URL=https://.../schedule.xlsx FACULTY_URL=https://.../faculty.csv
make docs
```

Downloads go to a temporary file and are moved into place only after curl --fail succeeds, so a failed download never overwrites good local data. The fresh workbook is newer than the CSV export, so make regenerates everything downstream automatically.

9. Testing

Test	Result
make test: 104 unit checks (time/day parsing incl. Excel 4:00:00 PM; CSV edge cases and header matching; loading the department format: ID split, session, async, AM→PM repair, TBA; alias matching; physical-room detection; session-aware conflicts; scheduler invariants: no office hours overlap any session's class + buffer, touching blocks merged, teaching-day default)	All pass
End-to-end: broken inputs in tests/data/ must exit 3 and report instructor and split-room double-booking, unknown instructor, lecturer hours overlapping a class, AM→PM repair, and TBA; a same-slot section in the other session must not be flagged	Pass
End-to-end: one input file instead of two must exit 1	Pass
AddressSanitizer + UndefinedBehaviorSanitizer on the CTEC data and the unit tests	No findings
Valgrind --leak-check=full on the CTEC data	No leaks, no errors
GCC -Wall -Wextra -Wpedantic -Wshadow -Wformat=2; Clang with -Wconversion	Zero warnings
Ruff (Python, 120-character lines)	Clean

10. Sample output

From output/schedule.txt:

Anthony Adedoyin (CT01)
Department: Computer Technology (CTEC) | Office hours location: Office TBA
Sections: 3 | Students enrolled: 53

CLASS SCHEDULE

Course	Session	Times	Location
CTEC 350.170 Prin & Meth of Intru Det & Pre	7R1	Monday & Wednesday 4:00 PM to 6:50 PM	108
CTEC 435.170 UNIX System Administration	7R1	Monday & Wednesday 7:00 PM to 9:50 PM	108
CTEC 350.180 Prin & Meth of Intru Det & Pre	7R2	Monday & Wednesday 4:00 PM to 6:50 PM	108

OFFICE HOURS (required 3.00 h/week, assigned 3.00 h/week; valid for both sessions)

Day(s)	Hours	Location	Arranged by
Monday	1:45 PM to 3:45 PM	Office TBA	Generated
Wednesday	2:45 PM to 3:45 PM	Office TBA	Generated

NOTE: Instructors are not to be interrupted during class times.

A section with repaired or missing data is marked with * and explained under the table:

CTEC 120.170 Prin of SecureCoding Using Jav	7R1	Monday & Wednesday 12:00 PM to 3:15 PM *	116
CTEC 125.555 Intro to Python programming	7R1	Online asynchronous (no set time)	Online
* CTEC 120.170: End time corrected from 3:15 AM to 3:15 PM			

The Word document opens with a **Data Issues** page, followed by one page per professor in the sample's table style. The Excel workbook has Summary, Classes, Office Hours and Data Issues sheets (errors highlighted), plus one sheet per professor. The text report also contains a weekly grid per professor showing which session each class belongs to (7R1, 7R2, or BOTH).

11. Limitations and future work

- The greedy scheduler is optimal per step, not globally; a constraint solver could balance hours across faculty.
- One office-hours schedule covers both sessions. For an instructor who teaches only in 7R2, a per-session schedule would free up the first half of the term.
- Office locations and contact details are not in the department data and must be completed by hand.
- The AM→PM repair assumes an end time earlier than the start is a data-entry error. It is always reported, never silent.

Appendix A - Submission contents

Path	Contents
Makefile	Build, library, csv, run, docs, test, inspect, fetch, clean
README.md	Compilation and run instructions
include/*.h	8 headers (7 modules + umbrella schedule.h)
src/main.c, src/lib/*.c	Driver + 7 library modules
build/libschedule.a	Static library (rebuilt by make)
data/CTEC_Fall_2026_Schedule.xlsx	Department class schedule (source)
data/class_schedule.csv	Input file 1 (CSV export of the workbook)
data/faculty.csv	Input file 2
scripts/xlsx_to_csv.py, scripts/export_docs.py, requirements.txt	Python steps
tests/	Unit tests and conflict test data
sample_output/	schedule.txt, CSV files, diagnostics.txt, .docx, .xlsx from a reference run
docs/report.pdf, docs/report.md	This report and its source